

Technical Architecture Document

This document details the software architecture, tech stack, and security model of the ZAIZEN platform.

System Architecture

ZAIZEN follows a decoupled, **Client-Server Architecture** with an N-Layer backend design.

Backend Layout (.NET 8 Core API)

- API Layer:** Controllers, Middleware, Action Filters (Authentication & Security).
- Application Layer:** MediatR Commands/Queries, Services for business logic (Payroll, Accounting, Tax).
- Domain Layer:** Core Entities, Interfaces, and Business Rules (OHADA Account constraints).
- Infrastructure Layer:** Data Access (EF Core), Repositories, External API integrations (DGI Client), and Storage.

Frontend Layout (React 18 + Vite)

- State Management:** React Context (Permissions, Company context).
- UI Library:** Vanilla CSS with custom "Premium Design System" (Glassmorphism).
- API Integration:** Axios-based wrapper for RESTful communication with JWT interception.

Technology Stack

Layer	Technology
Backend	.NET 8.0, ASP.NET Core
Database	PostgreSQL 16+
ORM	Entity Framework Core (Npgsql)
Frontend	React 18, TypeScript, Vite
Security	JWT (HS256), RBAC, WORM Audit Logs

Reporting	QuestPDF (PDF Generation), ClosedXML (Excel)
Infrastructure	Docker, Health Checks, MediatR

■ Security Model

1. Identity & Access Management

- **JWT Tokens:** Stateless authentication using secure signed tokens.
- **Role-Based Access Control (RBAC):** Fine-grained permissions (e.g. `journal:read`, `payroll:write`).
- **Company Isolation:** The `CompanyMembershipActionFilter` ensures users only interact with data belonging to their authorized companies.

2. Data Integrity (Immutable Audit)

- The `ImmutableAuditService` ensures every POST/PATCH/DELETE action is captured in an audit table.
- Financial records follow a "No-Delete" policy. Adjustments are made via reversal entries (storno), preserving the history of all transactions.

■ Data Schema Overview

- **Financials:** `JournalEntries` ↔ `JournalLines` ↔ `Accounts`
- **Commercial:** `SalesDocuments` ↔ `Products` ↔ `Warehouses`
- **Human Resources:** `Employees` ↔ `PayrollPeriods` ↔ `PayrollDetails`
- **CRM:** `Leads` ↔ `PipelineStages`

■ API Integration Patterns

- **RESTful endpoints:** standard CRUD operations.
- **Blob downloads:** PDF and Excel reports are streamed directly as binary blobs to the client.
- **Real-time validation:** The backend performs "Double-Entry" validation before committing any journal transaction.